artifact-1-proposed-architecture.md

Proposed Architecture

Summary

A serverless, event-driven AWS pipeline that processes PDFs up to 1,000 pages and 300 MB through seven durable stages. The application is AWS-native Document AI platform that accepts large PDFs, processes them asynchronously through a seven-stage pipeline, and delivers structured JSON extraction results through an API and webhook.

Core Decision: Wherever possible, deterministic code is to be used for every consequential decision, and only fallback to OCR extraction or Language Models when it won't suffice

Core Approach: A customer uploads a PDF directly to S3. The platform validates it, splits it into token-aware chunks, and processes each chunk in parallel bypassing OCR for pages that already have embedded text, running asynchronous OCR only for scanned or hybrid pages, and calling a language model only for schema-constrained extraction. Chunk results are merged deterministically. Every state transition is durable, every artifact is versioned, and every stage is replayable without duplicating work or charges.

The design uses AWS managed services throughout with a dedicated section for the self-hosted path which should be future scope once we have validated the rest of the plumbing of the system. All infrastructure is to be defined with AWS CDK in Python and deployed through Github Actions with quality gates.

2. Case Study Interpretation and Scope

The case study asks for a serverless, event-driven, API-driven platform capable of processing large PDFs at high throughput while controlling latency, cost, reliability, security, and MLOps risk.

What this design handles:

- PDF documents up to 1,000 pages and 300 MB.
- Document types: invoices, contracts, forms, and corporate reports.
- Sustained throughput of 10,000+ documents per day.

- Burst intake up to 50,000 documents, protected by queues, admission control, and per-tenant rate limits.
- P95 end-to-end latency under 5 minutes for accepted documents.
- Cost below $0.10 per 100 pages for a representative document mix.
- 99.9% platform availability through durable state, retries, DLQs, and replay.
- GDPR and SOC 2-aligned security, encryption, auditability, and erasure.
- Managed Bedrock inference in v1 with a defined self-hosted EKS path for high-volume economics.

Important scope clarifications:

- The 5-minute SLA clock starts at accepted_at, once the platform has received, validated, and malware-scanned the uploaded object. Upload time over customer networks is excluded.
- The $0.10 cost target is for a representative mix (~60% digital-text, ~30% hybrid, ~10% scanned). Fully-scanned or form-heavy documents will likely exceed the target.
- The case study lists Merging before AI Extraction. This design inverts the order because merge requires chunk-level extracted fields.

3. Assumptions

The following assumptions underpin the cost model, latency targets, and operational metrics. They are not defaults — changing them requires re-running the relevant calculations. (All prices are in USD)

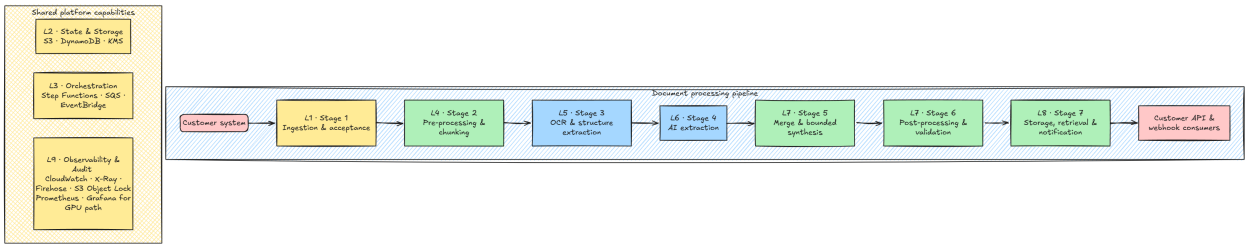
#	Area	Assumption
A1	Region	Primary AWS region is `us-east-1`.
A2	SLA clock	The 5-minute P95 clock starts at `accepted_at`. Upload time is excluded.
A3	File limits	PDFs up to 300 MB and 1,000 pages per document.
A4	Throughput	10,000+ docs/day sustained; 50,000-document burst.
A5	Document mix	~60% digital-text (OCR bypass), ~30% hybrid (selective OCR), ~10% fully scanned. Form-heavy workloads are cost-policy exceptions.
A6	Upload	Direct S3 presigned POST with Transfer Acceleration. 30-minute upload TTL. Customers never push files through Lambda.

#	Area	Assumption
A7	Malware scan	GuardDuty Malware Protection scan completes in seconds for typical files; budgeted at under 30 seconds worst case.
A8	Ingestion cost	S3 Transfer Acceleration 0.04/GB, GuardDuty 0.09/GB + 0.215/1K objects. Ingestion total: 0.04$ per 300 MB document
A9	Pre-processing	Stage 2 completes in under 30 seconds for typical digital/mixed PDFs. Lambda for ≤250 pages / ≤100 MB; ECR + Fargate above that.
A10	Token estimation	1 token ≈ 4 characters + 20% safety margin. Model-agnostic and conservative by design.
A11	OCR pricing	Assuming platform will exceed 1M Textract pages/month, qualifying for volume pricing: 0.0010/page (text/tables), 0.0040/page (forms/key-value).
A13	OCR concurrency	Starting defaults: 100 global active OCR units, 5 per tenant, 2 per job. Must be load-tested against Textract quotas before launch.
A14	Model pricing	Llama 3.1 8B on Bedrock: 0.22/1M tokens (input and output). Llama 3.1 70B: $0.72/1M tokens. On-demand rates.
A15	Chunk economics	Typical 100-page document → ~4 chunks × 19.2K tokens. Stage 4 cost: ~0.017/100 pages with 8B model. 70B recovery on all chunks: ~$0.055 which will exceed our budget. Must be used selectively.
A16	Token budget	≤25K target input, 30K hard cap, ≤1.5K output, ≤4K few-shot examples per chunk.
A17	Merge cost	Deterministic merge: ≤0.001/100 pages. Synthesis: ≤0.010/100 pages.
A18	Synthesis cap	≤20K input tokens, 30K hard cap, maximum 10 synthesis questions per document.
A19	Post-processing	Stage 6 cost: ≤0.001/100 pages. Hard stop-loss at $0.005 if retries
A20	Storage & delivery	Stage 7 cost: ≤0.001/100 pages storage + ≤$0.001/job for webhook and index.
A21	Webhook delivery	8 attempts max over 24 hours, exponential backoff with jitter, 5-second timeout per attempt.

#	Area	Assumption
A22	Signed URL TTL	15-minute default. Customer must re-call the API after expiry.
A23	Retention	Inputs and final outputs: 365-day default. Original inputs cold-tiered after 7 days. Enhanced images deleted after 24 hours. Rejected uploads: 7 days warm + 21 days cold.
A24	Self-hosted break-even	vLLM with INT8/FP8 on g5.xlarge (A10G) achieves ≥825 tokens/sec at 80% GPU utilization. Fixed infrastructure baseline: ~$1,453/month (2× reserved g5.xlarge + EKS overhead). Break-even: ~5,500–6,000 docs/day; recommended switch at ~10,000 docs/day for sufficient cost-savings margin.

4. Architecture Overview

The platform is structured as nine layers — six **pipeline layers** (L1, L4–L8), each producing a durable, versioned artifact that hands off to the next stage through an idempotent contract, and three **cross-cutting layers** (L2 state & storage, L3 orchestration, L9 observability & audit) that every pipeline stage depends on. Every pipeline stage is independently replayable from the previous stage's artifact.



Layer summary:

#	Layer	Key AWS services	Responsibility
L1	API & Admission (Stage 1)	API Gateway, WAF, Lambda authorizer, Submission & Validation Lambda, GuardDuty Malware Protection	Authenticate, create jobs, issue upload policies, validate uploads, gate on malware scan, enforce admission fairness
L2	State & Storage *(cross-cutting)*	S3 (input / intermediate / output / audit), DynamoDB, KMS	Durable artifact storage, job state, idempotency records, cryptographic tenant isolation

#	Layer	Key AWS services	Responsibility
L3	Orchestration *(cross-cutting)*	Step Functions Standard, Distributed Map, SQS, EventBridge	Coordinate stages, fan out chunks, buffer bursts, isolate failures via DLQs
L4	Pre-processing (Stage 2)	Lambda, ECS/Fargate	Profile PDF, classify pages, enhance scans, emit page and chunk manifests
L5	OCR & Structure (Stage 3)	Async Textract, Lambda normalizer	OCR scanned/hybrid pages, bypass OCR for digital text, produce citation-ready blocks and spans
L6	AI Extraction (Stage 4)	Bedrock Converse, Bedrock Guardrails	Extract schema-valid chunk JSON with mandatory citation grounding
L7	Quality Gates (Stages 5 + 6)	Lambda merge & validation workers, manual / degraded review queue	Merge chunk results, run schema / citation / business validators, normalize, redact, route unsafe outputs
L8	Delivery (Stage 7)	S3 output, API Gateway result endpoint, webhook dispatcher	Persist final result, expose signed URL, deliver pointer-only webhook notifications
L9	Observability & Audit *(cross-cutting)*	CloudWatch, X-Ray, EventBridge → Firehose → S3 Object Lock, Prometheus, Grafana	Trace, monitor, audit, alert, compare managed vs self-hosted inference

5. Requirements Traceability

The case study contains hard requirements across functional, non-functional, MLOps, and security domains. The table below shows how some important factors have been addressed.

Case study requirement	How the design addresses it
Serverless, API-driven, event-driven	Every stage is Lambda or Fargate triggered by events. No polling. Heavy PDFs use Fargate while simple work stays in Lambda.

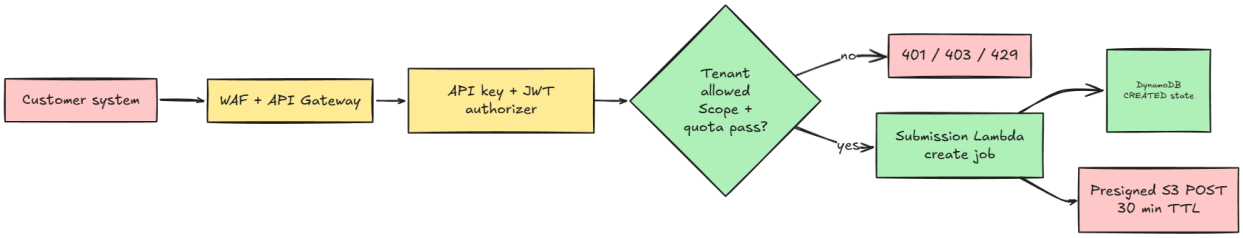
Case study requirement	How the design addresses it
Large PDF uploads (up to 300 MB)	Direct S3 presigned POST with Transfer Acceleration. File never passes through Lambda or API Gateway.
Documents up to 1,000 pages	Stage 1 validates page count before quota is consumed. Stage 2 uses Fargate for documents above the Lambda cutoff.
20–50 page chunks	Stage 2 chunks by page range, document structure, and token budget. Token-aware chunking prevents LLM context overflow.
Orientation/image enhancement	Stage 2 detects rotation, skew, and quality. Enhancement runs only on pages that need OCR
Asynchronous OCR	Stage 3 uses Textract async APIs with SNS -> SQS callbacks, bounded retries, DLQ, and SNS signature validation.
Text, forms, and tables	Direct text is default and Raw OCR is fallback. Structured Textract (Forms/Tables) only runs when required. Forms cost 0.0040/page vs $0.0010/page for text and tables
SLM/LLM for classification and entity extraction	Stage 4 uses Bedrock Llama 3.1 8B via Converse API with tool-use JSON output. 70B is used for when the extraction fails for critical fields.
128K context window limit	The system never places the full document in one prompt. Token-aware chunks with a 30K hard cap ensure every prompt fits safely within the model context, leaving headroom for schema, instructions, citations, and repair prompts.
Merge chunk outputs	Stage 5 merges chunk artifacts deterministically using schema-defined merge policies for each field.
Cross-page reasoning	Stage 5 synthesis is bounded to ≤10 targeted questions. The model receives only relevant candidates and source spans, not the full document.
Structured JSON output	Stage 6 produces `customer-result.json` as the customer-facing output. `final-result.json` and `validation-report.json` are platform-internal.
Job status and result retrieval API	`GET /v1/jobs/{id}` returns status and metadata. `GET /v1/jobs/{id}/result` returns a 15-minute signed URL for `customer-result.json` .

Case study requirement	How the design addresses it
Webhook without large payloads	Stage 7 sends HMAC-SHA256-signed pointer-only webhooks for all terminal states. No extracted values, raw text, PII, or presigned S3 URLs in webhook payloads.
10,000 docs/day sustained	Admission control, Distributed Map, per-tenant concurrency limits, and service quota guardrails along with the general serverless architecture
50,000-document burst	SQS and EventBridge decouple ingestion from processing. Tenant token buckets and per-stage concurrency limits prevent burst from overwhelming OCR, Bedrock, or DynamoDB.
Sub-5-minute P95 latency	Explicit stage budgets: Stage 2 ≤30s, Stage 3 P95 ≤180s, Stage 4 P95 ≤60s, Stage 5 P95 ≤30s, Stage 6 P95 ≤10s, Stage 7 P95 ≤10s. Queue age alarms trigger backpressure before SLA misses.
Less than $0.10 per 100-page document	Cost is controlled per stage. For representative-mix estimate: ingestion ~0.04, OCR ~0.01–0.03, Stage 4 extraction ~0.017, merge/synthesis ~0.011, post-processing and delivery ~$0.003. Scanned-heavy workloads are surfaced as exceptions via `cost_policy_result` tracking.
99.9% availability	Durable Step Functions state machine, SQS with DLQs, retry with exponential backoff and jitter, poison-pill isolation, and replay at document, chunk, OCR unit, model invocation, merge, and webhook scope.
Idempotency	Every asynchronous boundary has a scoped idempotency key incorporating job, chunk, artifact hash, policy version, and model route. Retries and replays cannot create duplicate billing events, OCR charges, or webhook deliveries.
Poison-pill handling	Repeated deterministic failures move to isolated poison queues with non-PII diagnostics, attempt count, failure reason codes, and operator replay paths.
KMS Keys encryption at rest	Every customer document, intermediate artifact, and output uses a per-tenant KMS key. Without the tenant key, artifacts are unreadable.

Case study requirement	How the design addresses it
365-day data retention	Stage 7 applies a 365-day default lifecycle policy. Original inputs are cold-tiered after 7 days. Tenant policy can shorten or extend within contract terms.
GDPR alignment	GDPR deletion workflow (Stages 1 and 7) deletes customer content and can crypto-shred via tenant key deletion. PII-free immutable audit records survive erasure by design.
SOC 2 alignment	Immutable audit via S3 Object Lock in compliance mode, CloudTrail API logging, CI/CD change-management evidence, and canary deployment controls.
PII protection in logs	Emit-time log scrubbing with pattern-based redaction. CloudWatch subscription filter feeds a Comprehend-backed inspection Lambda for leakage detection.
X-Ray distributed tracing	X-Ray traces propagate correlation ID, tenant ID, job ID, chunk ID, and stage across all Lambda, Fargate, Textract, Bedrock, and DynamoDB calls.
IaC with AWS CDK Python	All infrastructure: application, data, observability, security, and inference is defined in AWS CDK with Python and validated through CDK synth in CI.
GitHub Actions CI/CD with 80% coverage	GitHub Actions pipeline runs build, lint, unit tests (≥80% coverage), security checks, CDK synth validation, integration tests, and deployment.
Precision/Recall/F1 tracking	Stage 4 and Stage 5 evaluate document-level, field-level, and critical-field F1. Quality is tracked by document type, schema, page quality, OCR route, and model version.
10% canary rollouts with 1-hour soak	Canary releases are used for application code, prompts, schemas, merge policies, and model versions. Rollback criteria include critical-field F1, schema validation failure rate, cost spikes, and latency.
GPU orchestration (Karpenter/KEDA)	Section 7 describes the EKS self-hosted path with Karpenter GPU node provisioning and KEDA scaling from queue depth and vLLM request backlog.
Managed vs self-hosted break-even	Section 7 provides the full quantitative break-even analysis. The recommended transition point is ~10,000 docs/day.

6. Layer Details

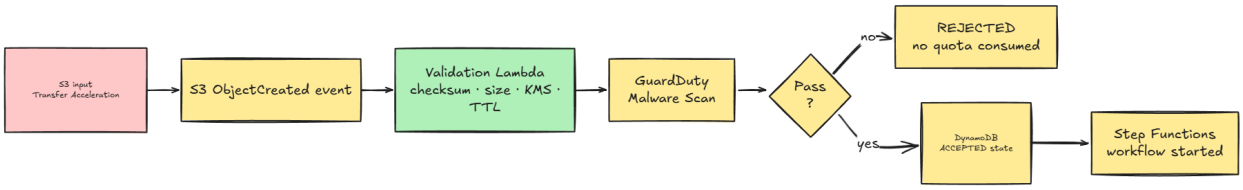
6.1 Authentication



Every job starts at the API boundary. WAF and API Gateway protect the public endpoint, the Lambda authorizer validates the API key and short-lived JWT. The API key identifies the tenant application then the JWT carries scopes such as `documents permissions, schema access, and upload permissions` .

Job creation and file upload are separated. After authentication, tenant policy, and quota checks pass, the customer calls `POST /v1/jobs` to get a pre-signed S3 upload target, then uploads the PDF directly to S3. The platform never proxies the file through Lambda.

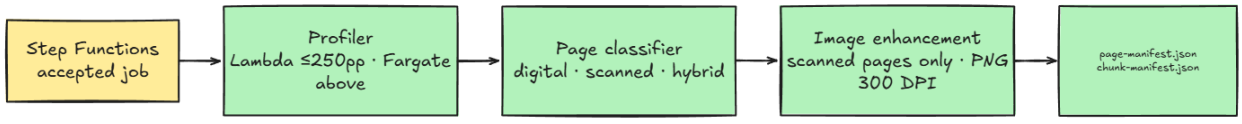
6.2 Ingestion and Acceptance



After upload, S3 triggers validation automatically. The Validation Lambda checks object ownership, checksum, content type, file size, page count, KMS key, upload TTL expiry, and the GuardDuty malware scan result. Only after all checks pass does the job move to `ACCEPTED` , tenant quota is consumed, and the Step Functions workflow starts.

Failed uploads do not consume quota. Tenant admission control returns `429` with retry guidance and `Retry-After` headers. Four separate rate limits protect different resources: job creation rate, pending upload count, accepts-per-minute (rate into expensive processing), and maximum concurrent active jobs.

6.3 Pre-processing

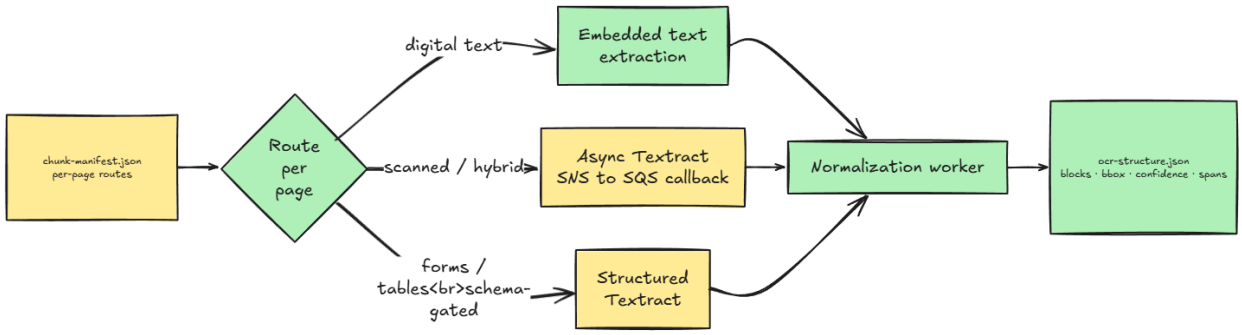


Stage 2 creates the manifests that drive all downstream parallelism. It runs in Lambda for smaller documents (≤250 pages / ≤100 MB) and ECS/Fargate for anything heavier.

The page classifier is deterministic in v1, it uses text density, image coverage, dictionary-word ratio, language confidence, orientation, and skew. Ambiguous pages default to the OCR path because missing a critical field is worse than paying for an unnecessary OCR call. A lightweight ML classifier is deferred to v2 once production-labeled examples are available.

Each chunk is bounded by both page count (20–50 pages) and token budget (25K target, 30K hard cap). This is the key control that ensures the downstream LLM never receives a prompt it cannot handle.

6.4 OCR and Structure Extraction



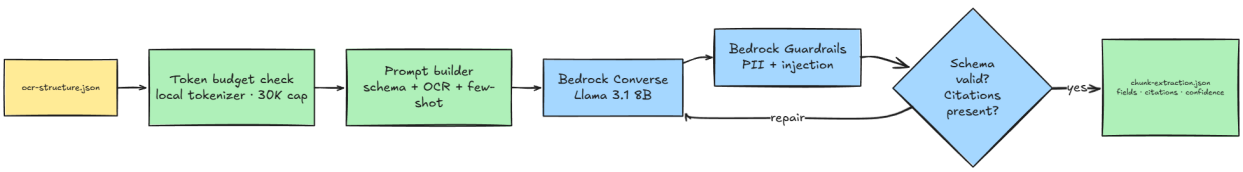
Stage 3 collects text from the cheapest reliable source for each page. Digital-text pages skip OCR entirely. Scanned and hybrid pages go through async Textract.

Structured OCR (Forms/Tables) is only triggered when required by schema. At 0.0040/page for form extraction vs 0.0010/page for text, enabling structured OCR by default on all pages would consume the entire $0.10 budget on Textract alone before the LLM is even called.

Every Textract call uses `OutputConfig` to write results to the tenant-scoped S3 prefix with the tenant KMS key. Without this, Textract may write outside the tenant isolation boundary. Callbacks are validated against SNS signing certificates, expected topic ARN, job ID, tenant/chunk match, and a freshness window.

The output `ocr-structure.json` preserves page number, reading order, block IDs, bounding boxes, confidence scores, and source spans. These are the citation anchors for everything downstream.

6.5 AI Extraction

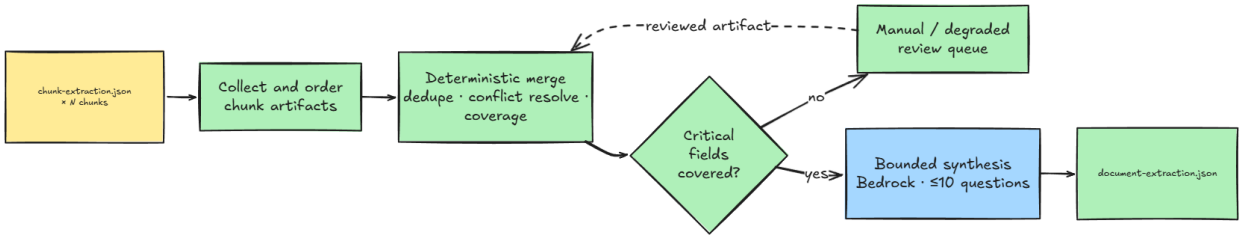


Stage 4 uses Bedrock Llama 3.1 8B with Converse tool-use to produce one schema-valid JSON artifact per chunk. The Converse tool-use model (`submit_extraction` tool generated from the tenant JSON Schema) forces structured output, therefore, free-form model text is ignored.

The core hallucination control is citation enforcement. Every non-null extracted field must carry a `source_citations` array pointing to a specific page, block, and character span from `ocr-structure.json` . If a value has no valid citation, it is rejected and marked `not_found` . The model can say a field is absent, but it cannot invent a value to satisfy a required schema field.

If schema validation or citations fail, the system retries once with a compact repair prompt. If critical fields still fail, the chunk routes to Llama 3.1 70B recovery. If that fails, it routes to manual inspection.

6.6 Merge and Document-Level Synthesis

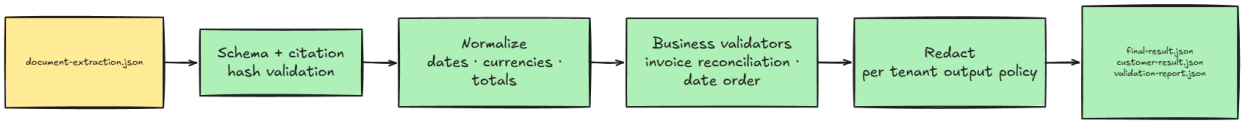


Stage 5 merges chunk artifacts using schema-defined merge policies. Every field declares its cardinality, criticality, merge strategy, and conflict policy. The merge is deterministic first, collecting candidates, deduplicating, normalizing, and applying the field strategy without touching the model.

Invoice total conflicts, critical singleton conflicts, and uncited critical fields always go to manual inspection. The model is not allowed to guess a critical value that multiple high-confidence candidates disagree on.

Bounded synthesis is used only when the schema declares it or Stage 4 flagged a cross-chunk dependency. The synthesis prompt contains only the relevant candidates and source spans, never the full document. Maximum 10 synthesis questions per document, with overflow routed to manual inspection.

6.7 Post-processing

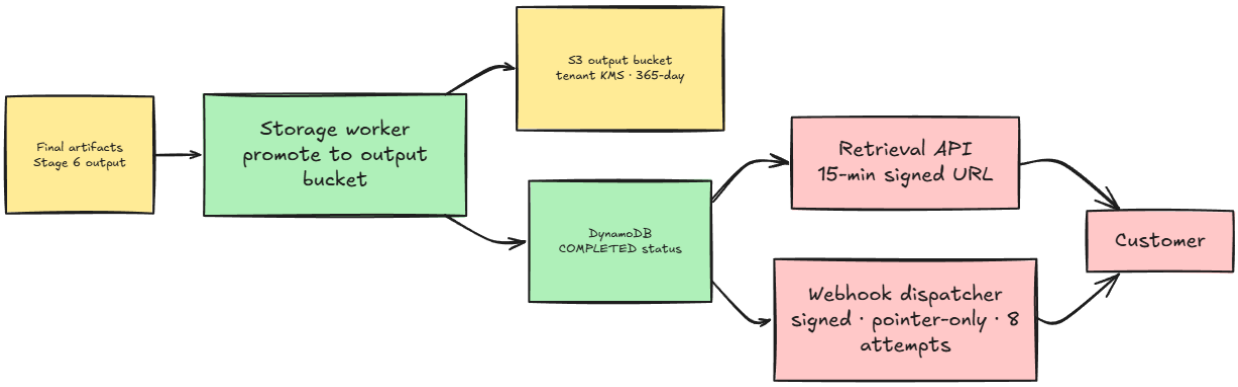


Stage 6 is entirely deterministic. It validates the merged document against the tenant JSON Schema, normalizes field values (dates, currencies), runs business validators (invoice reconciliation, date ordering, signature presence), applies tenant redaction rules, and packages the three output artifacts.

Stage 6 does not call the model. If it detects a recoverable issue, it emits one bounded replay request to Stage 4 or Stage 5, maximum one replay per target stage per job. Recurring failures route to manual inspection rather than triggering unbounded automated retry loops.

Redaction runs after validation, never before. Validators need real values to check totals and identifiers. If redaction fails for any field, the entire output is blocked, unredacted data is never delivered to customers.

6.8 Storage, Retrieval, and Notification



Stage 7 promotes Stage 6 artifacts to the durable output bucket, updates the terminal job state, and triggers webhook delivery. The terminal state is marked only after `customer-result.json` is durably written and indexed.

Retrieval always returns a short-lived signed URL, never an inline result body. This keeps the API response shape consistent whether the result is 10 KB or 10 MB. Webhooks are pointer-only, they contain a retrieval URL, a hash, and status metadata, not extracted values. Webhook delivery is decoupled from the job result: a customer endpoint being down does not roll back a completed extraction.

6.9 Cross-Cutting Controls

Idempotency:

Stage	Idempotency key
Job creation	`tenant_id + client_request_id + request_hash`
Upload validation	Object key hash -> one job; duplicate S3 events are no-ops

Stage	Idempotency key
OCR	`tenant_id + job_id + chunk_id + ocr_unit_id + input_artifact_hash`
AI extraction	`tenant_id + job_id + chunk_id + extraction_unit_id + schema_version + prompt_version + model_route + input_artifact_hash`
Merge	`tenant_id + job_id + schema_version + merge_policy_version + ordered_chunk_artifact_hashes`
Post-processing	`tenant_id + job_id + schema_version + post_processing_policy_version + stage5_artifact_hash + output_policy_hash`
Storage/webhook	`tenant_id + job_id + post_processing_attempt_id + final_result_hash`

Multi-tenancy and isolation:

- Tenant identity is resolved from API key and JWT claims.
- S3 object keys are platform-generated and tenant/job-scoped. Customers cannot influence key structure.
- Per-tenant KMS keys provide cryptographic isolation: no cross-tenant artifact is readable without the tenant's key.
- DynamoDB uses high-cardinality `JOB#<job_id>` partition keys with write-sharded tenant GSIs to avoid hot partitions during bursts.
- Schemas, prompts, quotas, cost ledger, and metrics are all scoped by tenant.

Rate limiting and backpressure:

- WAF rate limits protect public endpoints.
- API Gateway and Lambda authorizer enforce per-tenant rate limits at job creation.
- Stage 3 and Stage 4 have separate global, per-tenant, and per-job concurrency limits for OCR and Bedrock calls.
- Queue age is an SLA risk signal: when queue age threatens the P95 budget, the platform applies admission backpressure and returns `429 Retry-After` for new work.

Security and compliance:

- S3 denies non-TLS access with `aws:SecureTransport=false`. API Gateway uses TLS 1.2+.
- All audit events flow through EventBridge and Firehose to S3 Object Lock in compliance mode. Audit records store only operational facts, hashes, and status

codes, never document text or PII.
- GDPR erasure deletes or crypto-shreds customer content while preserving the PII-free audit trail, which is required for SOC 2.

Reliability:

- Every asynchronous boundary uses retry with exponential backoff and jitter.
- DLQs isolate failed work with stage, tenant, job, attempt count, failure reason, and artifact pointers.
- Replay is defined at document, chunk, OCR unit, model invocation, merge, validation, and webhook scope.
- Platform-initiated recovery is non-billable. Customer-requested replay after input or policy changes is billable.

MLOps:

- All infrastructure is CDK Python. GitHub Actions enforces ≥80% unit coverage, CDK synth validation, and integration tests on the core job lifecycle.
- Prompts, schemas, merge policies, post-processing policies, and model routes are versioned and independently rollback-able.
- Model quality is evaluated by document-level, field-level, and critical-field precision/recall/F1, tracked by document type, schema, page quality, OCR route, and model version.
- Canary releases use 10% traffic over a 1-hour soak. Rollback criteria include critical-field F1 regression, schema validation failure rate, cost spikes, latency, and manual-inspection rate increase.

7. Self-Hosted LLM Option

7.1 Infrastructure Design

The platform uses three GPU node tiers managed by Karpenter:

Tier	Instance	Effective rate	Role
Reserved baseline	`g5.xlarge` , 1-year reserved	~$0.634/hr	Sustained demand, right-sized to high effective utilization
Spot burst	`g5.xlarge` spot	Market price	Elastic surge when reserved capacity is saturated; not assumed cheaper until measured

Tier	Instance	Effective rate	Role
On-demand overflow	`g5.xlarge`	~$1.006/hr	Last resort when spot is unavailable or interrupted

Karpenter manages node provisioning with a GPU node pool capped at a configured ceiling as a cost guardrail. Nodes that have been empty for 5 minutes are drained automatically. KEDA scales the vLLM deployment from two signals: the SQS extraction queue depth (one message = one chunk awaiting model inference) and the vLLM internal `num_requests_waiting` metric from Prometheus (requests queued inside the model server's KV-cache scheduler)

vLLM serves the default 8B model in **FP16** for parity with the managed model path. The self-hosted cluster must also preserve the same model contract as Bedrock: schema-constrained output, citation grounding, PII controls, prompt-injection defenses, model-version tracking, and canary/rollback support.

7.2 Observability

Self-hosted metrics are collected by Prometheus and visualized in Grafana. They are normalized to the same `cost_per_1m_tokens` unit used for Bedrock, so the cost comparison panel shows both paths on the same axis in real time.

GPU and infrastructure metrics:

Metric	Source	Purpose
`DCGM_FI_DEV_GPU_UTIL`	DCGM Exporter	GPU compute utilization per device
`DCGM_FI_DEV_FB_USED/FREE`	DCGM Exporter	VRAM used vs free
`node_provisioning_time_seconds`	Karpenter	Time from scale trigger to node ready
`spot_interruption_count`	CloudWatch Events	Spot reclaim rate per instance type
`gpu_node_hours`	Kubernetes / AWS CUR	Billed GPU-hours by reserved, spot, and on-demand tier

vLLM inference metrics:

Metric	Purpose
`vllm:num_requests_running`	Active concurrent requests
`vllm:num_requests_waiting`	Internal queue depth
`vllm:gpu_cache_usage_perc`	KV cache hit rate
`vllm:time_to_first_token_seconds`	TTFT
`vllm:prompt_tokens_total` / `vllm:generation_tokens_total`	Token throughput
`stage4_extraction_success_rate`	Confirms self-hosted output quality does not regress versus Bedrock
`schema_validation_failure_rate`	Detects output-contract regressions
`citation_validation_failure_rate`	Detects grounding regressions and hallucination risk

7.3 Break-Even Analysis

Let:

```
P_b = Bedrock price per 1M tokens for the default 8B path
H_g = hourly cost for one GPU node
E = effective sustained billed throughput in tokens/sec
```

Effective throughput `E` is real utilization. If a GPU can process `T` tokens/sec under benchmark load but is only usefully busy `U` of the time, then:

```
E = T × U
```

Self-hosted cost per 1M tokens is:

```
cost_per_1M = H_g / (E × 3600 / 1,000,000)
```

Self-hosted beats Bedrock only when:

```
H_g / (E × 3600 / 1,000,000) < P_b
```

Rearranged:

```
E > H_g × 1,000,000 / (P_b × 3600)
```

Using the current planning inputs:

```
H_g = $0.634/hr for 1-year reserved g5.xlarge
P_b = $0.22 / 1M tokens for Bedrock Llama 3.1 8B

E > 0.634 × 1,000,000 / (0.22 × 3600)
E > ~801 effective tokens/sec
```

It is the **minimum measured effective throughput** required for one reserved `g5.xlarge` to match Bedrock's token price before adding EKS control-plane, observability, engineering, and operational overhead.

If the target operating utilization is 80%, the measured raw serving throughput must be higher:

```
T > 801 / 0.80
T > ~1,001 tokens/sec
```

If observed throughput or utilization falls below that line, Bedrock remains cheaper on pure token economics.

8. Pricing and Latency Summary

This section consolidates the per-stage latency and cost budgets that the architecture enforces, with the assumption that grounds each number. All cost figures are for a **representative document mix** (60% digital-text, 30% hybrid, 10% scanned per A5) at the **per-100-pages** unit; ingestion is also shown per-document because it scales with file size, not page count.

8.1 Per-Stage Latency and Cost

Stage	Latency budget	Cost / 100 pages	Primary cost drivers	Assumptions
Ingestion & acceptance	Excluded from SLA clock	~0.005 typical · ~$0.04 worst-case (per **document**,	S3 Transfer Acceleration (0.04/GB) + GuardDuty Malware Protection (0.09/GB + $0.215/1K objects)	A2 (clock starts at `accepted_at`); A6 (presigned POST, no Lambda in data path); A8

Stage	Latency budget	Cost / 100 pages	Primary cost drivers	Assumptions
		not per 100 pages)		($0.04 is the 300 MB ceiling)
Pre-processing	≤ 30 s typical	< $0.001	Lambda metadata path (≤ 250 pp / ≤ 100 MB) or Fargate vCPU-seconds for image enhancement	A9 (compute path split); deterministic page classifier, no model calls
OCR & structure	P95 ≤ 180 s	0.010 – $0.040	Async Textract `DetectDocumentText` at $0.0010/page (volume tier)	A5 (60/30/10 mix); A11 (volume tier above 1 M pages/month). Structured OCR (Forms/Tables) is schema-gated
AI extraction	P95 ≤ 60 s	0.013 – $0.017	Bedrock Llama 3.1 8B tokens at $0.22/1M (input + output)	A14 (8B rate); A15 (4 chunks × ~19.2K tokens for ~100 pages); A16 (token budget per chunk). Assumes ≤ 5% chunk-level 70B recovery rate
Merge & synthesis	P95 ≤ 30 s	0.005 – $0.011	Deterministic merge compute (~$0.001) + bounded 8B synthesis when triggered	A17 (cost caps); A18 (≤ 10 synthesis questions per doc, ≤ 20 K input tokens total). Synthesis fires on contracts/reports, rarely on invoices
Post-processing	P95 ≤ 10 s	< $0.001	Lambda compute for schema validation, normalization, redaction	A19 (deterministic only, no model inference). Hard stop-loss at

Stage	Latency budget	Cost / 100 pages	Primary cost drivers	Assumptions
				$0.005 signals retry storm or config bug
Storage & delivery	P95 ≤ 10 s	~$0.002	S3 PUT (3 output artifacts) + DynamoDB terminal update + webhook delivery	A20 (storage + delivery split); A21 (8-attempt webhook retry); A22 (15-min signed URL). 365-day retained-storage is tracked separately from per-job processing cost

8.2 Latency composition

The per-stage P95 budgets sum to `30 + 180 + 60 + 30 + 10 + 10 = 320 s` -> over the 300 s SLA at face value. This is intentional headroom: P95 latencies do not add linearly. A document does not hit the 95th percentile on every stage simultaneously, so end-to-end P95 is closer to median.

Per-stage budgets also act as **backpressure tripwires**: when queue age in any stage threatens its budget, admission control returns `429 Retry-After` for new work *before* the end-to-end SLA is at risk.

8.3 Cost composition

The 0.036 – 0.061 per-100-pages range assumes the representative mix in A5 and leaves $0.04 – $0.06 of headroom against the $0.10 target. The headroom left can then take:

- 70B recovery on a fraction of chunks (each 8B → 70B chunk swap adds ~$0.014 at the A14 rate)
- Textract volume tier not applying (+ 50% on OCR, ~ +$0.010)
- Denser-than-typical content (higher tokens per chunk than A15's 19.2 K)